1. What does AJAX stand for in client-side web development?

(A) Asynchronous JavaScript and XML
(B) Advanced JavaScript and XHTML
(C) Automated JSON and XML
(D) Asynchronous Java and XHTML

2. The core advantage of using AJAX in web applications is:

(A) Updating portions of a webpage dynamically with server data without requiring a full page refresh
(B) Executing code on multiple CPU cores
(C) Compressing images automatically
(D) Writing server backend logic in CSS

3. Which legacy browser object provided the original foundation for AJAX requests before the modern Fetch API?

(A) XMLHttpRequest (XHR)
(B) WebClient
(C) HttpRequest
(D) SocketClient

4. What data interchange format has predominantly replaced XML in modern AJAX web applications due to its lightweight syntax?

(A) JSON (JavaScript Object Notation)
(B) YAML
(C) CSV
(D) Binary ASN.1

5. In an XMLHttpRequest object, which method initializes a request with HTTP method and URL?

(A) xhr.open(method, url, async)
(B) xhr.send(data)
(C) xhr.init(url)
(D) xhr.connect(url)

6. Which method sends the HTTP request across the network to the server?

(A) xhr.send()
(B) xhr.open()
(C) xhr.dispatch()
(D) xhr.execute()

7. Which XMLHttpRequest property holds the numeric HTTP response status code (e.g., 200, 404)?

(A) xhr.status
(B) xhr.code
(C) xhr.statusCode
(D) xhr.readyState

8. Which property tracks the state of an XMLHttpRequest from 0 (UNSENT) to 4 (DONE)?

(A) xhr.readyState
(B) xhr.status
(C) xhr.phase
(D) xhr.state

9. What readyState value indicates that the request operation is completely finished and response data is ready?

(A) 4 (DONE)
(B) 3 (LOADING)
(C) 2 (HEADERS_RECEIVED)
(D) 1 (OPENED)

10. Which event handler on XMLHttpRequest is called whenever the readyState property changes?

(A) xhr.onreadystatechange
(B) xhr.onload
(C) xhr.onfinish
(D) xhr.oncomplete

11. Which modern browser native method returns a Promise that resolves to the Response to a network request?

(A) fetch(url, options)
(B) http.get(url)
(C) ajax.request(url)
(D) window.request(url)

12. What does a JavaScript Promise represent?

(A) The eventual completion (or failure) of an asynchronous operation and its resulting value
(B) A multi-threaded thread pool
(C) An immutable database record
(D) A compiler directive

13. What are the three mutually exclusive states a Promise can exist in?

(A) Pending, Fulfilled, Rejected
(B) Starting, Running, Finished
(C) Open, Loading, Closed
(D) Active, Inactive, Broken

14. Which method on a Promise registers callbacks to handle its fulfilled value or rejection reason?

(A) promise.then(onFulfilled, onRejected)
(B) promise.done()
(C) promise.resolve()
(D) promise.complete()

15. Which method on a Promise registers a callback to handle errors and rejections exclusively?

(A) promise.catch(onRejected)
(B) promise.error()
(C) promise.fail()
(D) promise.handle()

16. Which method on a Promise registers a callback that executes unconditionally when the Promise settles, regardless of success or failure?

(A) promise.finally(callback)
(B) promise.always()
(C) promise.end()
(D) promise.settle()

17. What syntax introduced in ES2017 provides clean, synchronous-looking syntax for working with Promises?

(A) async / await
(B) try / catch
(C) yield / next
(D) defer / return

18. The 'await' keyword can strictly only be used inside:

(A) Functions declared with the 'async' keyword (or top-level modules in modern JS)
(B) Any standard for loop
(C) Arrow functions without async
(D) Global scripts in HTML4

19. What does an 'async' function always return automatically?

(A) A Promise (wrapping the returned value)
(B) A Generator
(C) An XMLHttpRequest object
(D) Undefined

20. How are errors caught and handled when using async/await syntax in JavaScript?

(A) Using standard try...catch blocks
(B) Using .catch() chaining only
(C) Using onError listeners
(D) Errors cannot be caught in async functions

21. When calling `fetch(url)`, does a 404 Not Found or 500 Internal Server Error cause the returned Promise to reject?

(A) No: the Promise resolves normally; it only rejects on network failures or CORS blocks (check `response.ok`)
(B) Yes: all 4xx and 5xx reject automatically
(C) Yes: only 500 rejects
(D) It throws a fatal SyntaxError

22. Which property on the Fetch API `Response` object is true if the HTTP status code is in the range 200?299?

(A) response.ok
(B) response.success
(C) response.valid
(D) response.isDone

23. Which asynchronous method on the Fetch `Response` parses the response body as JSON, returning a Promise?

(A) response.json()
(B) response.toJson()
(C) response.parse()
(D) JSON.parse(response)

24. Which method on the Fetch `Response` extracts the response payload as a plain text string?

(A) response.text()
(B) response.toString()
(C) response.string()
(D) response.body()

25. What is jQuery in client-side web development?

(A) A lightweight, cross-platform JavaScript library designed to simplify HTML DOM tree traversal, manipulation, event handling, and AJAX
(B) A server-side programming language
(C) A replacement for CSS
(D) A database engine

26. What is the global factory function / shortcut identifier used to access jQuery?

(A) $ (dollar sign) or jQuery
(B) # (hash)
(C) @ (at)
(D) & (ampersand)

27. How do you select all `<p>` elements with the class 'intro' in jQuery?

(A) $('p.intro')
(B) $('p #intro')
(C) $('.intro p')
(D) jQuery.select('p.intro')

28. Which jQuery method is used to ensure code executes only after the entire DOM is ready for manipulation?

(A) $(document).ready(function() { ... }) or $(function() { ... })
(B) $(window).load()
(C) $(document).start()
(D) $(body).ready()

29. Which jQuery method is used to hide selected elements by setting display: none?

(A) $('selector').hide()
(B) $('selector').vanish()
(C) $('selector').remove()
(D) $('selector').invisible()

30. Which jQuery method is used to reveal hidden elements by restoring their display property?

(A) $('selector').show()
(B) $('selector').display()
(C) $('selector').visible()
(D) $('selector').appear()

31. Which jQuery method toggles the visibility of selected elements (hiding if shown, showing if hidden)?

(A) $('selector').toggle()
(B) $('selector').switch()
(C) $('selector').flip()
(D) $('selector').alter()

32. Which jQuery animation method smoothly fades an element in by increasing its opacity to 1?

(A) $('selector').fadeIn()
(B) $('selector').fadeUp()
(C) $('selector').fadeVisible()
(D) $('selector').appear()

33. Which jQuery animation method smoothly slides an element down into view?

(A) $('selector').slideDown()
(B) $('selector').slideIn()
(C) $('selector').slideOpen()
(D) $('selector').scrollDown()

34. Which jQuery method sets or gets the text content of selected elements (equivalent to textContent)?

(A) $('selector').text()
(B) $('selector').val()
(C) $('selector').html()
(D) $('selector').content()

35. Which jQuery method sets or gets the HTML content inside selected elements (equivalent to innerHTML)?

(A) $('selector').html()
(B) $('selector').text()
(C) $('selector').inner()
(D) $('selector').markup()

36. Which jQuery method sets or gets the value of form input fields (e.g. textboxes, selects)?

(A) $('selector').val()
(B) $('selector').value()
(C) $('selector').text()
(D) $('selector').input()

37. Which low-level jQuery AJAX method performs an asynchronous HTTP request with configurable settings?

(A) $.ajax(settings)
(B) $.fetch()
(C) $.http()
(D) $.request()

38. Which shorthand jQuery AJAX method executes an HTTP GET request to load data from the server?

(A) $.get(url, data, successCallback)
(B) $.post()
(C) $.load()
(D) $.fetch()

39. Which shorthand jQuery AJAX method sends data to the server using an HTTP POST request?

(A) $.post(url, data, successCallback)
(B) $.get()
(C) $.send()
(D) $.submit()

40. Which shorthand jQuery method loads HTML content from a remote server URL and inserts it directly into the selected DOM element?

(A) $('selector').load(url)
(B) $('selector').import()
(C) $('selector').fetch()
(D) $('selector').insert()

41. What was the critical code architecture problem colloquially known as 'Callback Hell' (or the Pyramid of Doom)?

(A) Nesting deeply recursive asynchronous callbacks inside other callbacks, making error handling, debugging, and sequential flow unreadable
(B) A stack overflow caused by infinite loops
(C) A browser crash from memory exhaustion
(D) Slow network connections

42. How do Promises resolve the Callback Hell problem?

(A) By allowing asynchronous operations to be flattened into readable linear chains using .then() and centralized error handling via .catch()
(B) By converting JavaScript to C++
(C) By making code synchronous and blocking
(D) By executing callbacks on the GPU

43. What does `Promise.all([p1, p2, p3])` do when passed an array of Promises?

(A) Fulfills when ALL input promises fulfill, returning an array of resolved values; rejects IMMEDIATELY if any single promise rejects (all-or-nothing)
(B) Waits for the fastest promise only
(C) Returns settled states for all promises
(D) Executes promises sequentially

44. What does `Promise.allSettled([p1, p2, p3])` do compared to `Promise.all()`?

(A) Waits for ALL input promises to settle (either fulfill or reject), returning an array of descriptor objects: `{ status: 'fulfilled', value }` or `{ status: 'rejected', reason }`
(B) Rejects on the first failure
(C) Only returns fulfilled values
(D) Cancels pending requests

45. What does `Promise.race([p1, p2, p3])` do?

(A) Settles (fulfills or rejects) as soon as the FIRST promise among the input promises settles, adopting that promise's value or reason
(B) Waits for all to fulfill
(C) Returns the slowest promise
(D) Runs promises in parallel threads

46. What does `Promise.any([p1, p2, p3])` do?

(A) Fulfills as soon as the FIRST input promise FULFILLS successfully; if all reject, it rejects with an AggregateError
(B) Rejects if any promise rejects
(C) Waits for all to finish
(D) Selects a random promise

47. What is the purpose of the Same-Origin Policy (SOP) enforced by web browsers?

(A) A critical security mechanism restricting scripts on one origin from accessing or reading resources from a different origin (domain, protocol, port)
(B) Prevents users from visiting foreign websites
(C) Restricts CSS animations
(D) Blocks all HTTP requests

48. Which three components define a web 'Origin' under the Same-Origin Policy?

(A) Protocol (scheme, e.g. https), Hostname (domain), and Port number (e.g. :443)
(B) Domain name and path only
(C) IP address and MAC address
(D) URL and query parameters

49. What mechanism allows a web server to explicitly permit cross-origin requests from browsers?

(A) Cross-Origin Resource Sharing (CORS)
(B) Content Security Policy (CSP)
(C) Domain Name System (DNS)
(D) SameSite Cookies

50. Which HTTP response header sent by the server specifies which external origins are permitted to access the resource?

(A) Access-Control-Allow-Origin
(B) Access-Control-Allow-Methods
(C) Access-Control-Allow-Headers
(D) Cross-Origin-Permit

51. What is a CORS 'Preflight Request'?

(A) An automatic preliminary HTTP OPTIONS request dispatched by the browser before the actual request to verify that the server permits the custom method or headers
(B) A DNS lookup request
(C) A ping test to measure latency
(D) A WebSocket handshake

52. Under what conditions does a browser trigger a CORS Preflight OPTIONS request?

(A) When the request uses HTTP methods other than GET, POST, or HEAD, or sets custom headers (like Authorization), or uses Content-Type other than urlencoded, form-data, or text/plain
(B) On every single GET request
(C) Only when cookies are enabled
(D) Only over HTTPS

53. Which header must be included in a Fetch request to transmit and include cross-origin authentication cookies?

(A) `credentials: 'include'`
(B) `credentials: 'same-origin'`
(C) `auth: true`
(D) `cookies: 'allow'`

54. How do you send a POST request with a JSON payload using the modern Fetch API?

(A) `fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) })`
(B) `fetch(url, 'POST', data)`
(C) `fetch.post(url, data)`
(D) `fetch(url, { data })`

55. What is the WebSocket API in HTML5 and JavaScript?

(A) A protocol providing full-duplex, bidirectional, persistent communication channels over a single TCP socket connection between client and server
(B) A replacement for HTTP GET
(C) A tool to download video streams
(D) A local browser database

56. How is a persistent WebSocket connection initiated in JavaScript?

(A) const socket = new WebSocket('wss://example.com/socket');
(B) const socket = new Socket('http://...');
(C) const socket = WebSocket.connect();
(D) const socket = fetch.socket();

57. Which event handler on a WebSocket instance is invoked when data arrives from the server?

(A) socket.onmessage = (event) => { console.log(event.data); };
(B) socket.ondata
(C) socket.onreceive
(D) socket.onread

58. How do you transmit data to the server across an open WebSocket connection?

(A) socket.send(payload);
(B) socket.emit(payload);
(C) socket.post(payload);
(D) socket.write(payload);

59. How do WebSockets differ fundamentally from HTTP polling or long-polling?

(A) WebSockets establish a permanent open TCP channel with minimal 2-byte framing overhead; polling repeatedly opens new HTTP connections with bulky headers
(B) WebSockets only send images
(C) HTTP polling is faster
(D) WebSockets are unidirectional

60. What is Server-Sent Events (SSE) using EventSource, and how does it differ from WebSockets?

(A) SSE provides lightweight, unidirectional (Server-to-Client) real-time streaming over standard HTTP; WebSockets provide full bidirectional messaging
(B) SSE is bidirectional; WebSockets are unidirectional
(C) SSE requires binary UDP protocols
(D) SSE is deprecated

61. How does jQuery method chaining work (e.g., `$('#box').fadeIn().css('color', 'red').slideUp();`)?

(A) Most jQuery manipulation methods return the jQuery object instance (`this`), allowing subsequent methods to be called immediately in sequence
(B) By executing methods in parallel threads
(C) Using Promise.all under the hood
(D) Through JavaScript prototypes only

62. What does the jQuery method `$.param(object)` do?

(A) Serializes an object or array into a standard URL query string format suitable for HTTP GET/POST parameters
(B) Converts object to JSON
(C) Calculates object memory size
(D) Validates object keys

63. In jQuery, what does `event.preventDefault()` accomplish compared to `return false` inside an event handler?

(A) `preventDefault()` halts default browser action; `return false` in jQuery halts default action AND calls `stopPropagation()` simultaneously
(B) They are identical in all JavaScript
(C) `return false` only works in React
(D) preventDefault throws an exception

64. How do you perform Event Delegation in jQuery?

(A) `$('#parent').on('click', '.child-selector', handler);`
(B) `$('.child-selector').delegate('click', handler);`
(C) `$('#parent').delegateClick('.child', handler);`
(D) `$.delegate('#parent', '.child', handler);`

65. What does the method `Promise.resolve(value)` do?

(A) Returns a Promise object that is already fulfilled with the given value
(B) Forces a promise to re-execute
(C) Clears a promise queue
(D) Converts a promise to a synchronous value

66. What does the method `Promise.reject(reason)` do?

(A) Returns a Promise object that is rejected with the given reason or error
(B) Aborts all active network requests
(C) Disables promise catching
(D) Throws a fatal JavaScript error

67. In async/await, what happens if an awaited Promise rejects and there is no surrounding try...catch block?

(A) The unhandled rejection causes the async function's returned Promise to reject, triggering an 'UnhandledPromiseRejection' warning/error
(B) The browser reloads the page
(C) The program pauses indefinitely
(D) It silently sets the variable to null

68. How can you implement a sleep/delay function in modern JavaScript using Promises and async/await?

(A) const sleep = ms => new Promise(resolve => setTimeout(resolve, ms)); await sleep(2000);
(B) window.sleep(2000);
(C) thread.sleep(2000);
(D) await setTimeout(2000);

69. What is the difference between sequential execution and parallel execution when using async/await?

(A) Awaiting promises sequentially in a loop pauses on each step; firing promises first and awaiting with `Promise.all()` executes them concurrently
(B) Parallel execution requires 4 CPU cores
(C) Sequential execution is faster
(D) Parallel execution cannot catch errors

70. What is JSONP (JSON with Padding), historically used to bypass Same-Origin Policy before CORS?

(A) A legacy technique where a `<script>` tag was dynamically inserted to request data wrapped in a client-side callback function execution
(B) An encrypted JSON format
(C) A compression tool for JSON
(D) A protocol for binary sockets

71. Why was JSONP vulnerable to severe security risks?

(A) It executes arbitrary JavaScript received from a third-party server directly with full page privileges, exposing sites to remote script injection (XSS)
(B) It drained device battery
(C) It bypassed SSL certificates
(D) It disabled HTML forms

72. What does the `AbortController` API allow you to do with a Fetch API request?

(A) Cancel an ongoing in-flight HTTP request programmatically via its `signal` property
(B) Pause and resume a download midway
(C) Speed up download bandwidth
(D) Encrypt request payloads

73. Which modern browser API provides low-level streaming access to fetch response bodies as chunks arrive over the wire?

(A) Streams API (ReadableStream via `response.body.getReader()`)
(B) WebSocket API
(C) FileReader API
(D) ChunkParser API

74. How do you upload a file with additional form metadata using Fetch and FormData?

(A) const fd = new FormData(); fd.append('file', fileInput.files[0]); fd.append('user', 'John'); fetch(url, { method: 'POST', body: fd });
(B) Send as JSON string
(C) Encode file as hex string in URL
(D) Use HTTP GET with file query param

75. Why should you NEVER manually set the `Content-Type: multipart/form-data` header when sending FormData via Fetch?

(A) The browser must set it automatically along with the required unique multipart boundary string (e.g., boundary=----WebKitFormBoundary...)
(B) Fetch does not support multipart
(C) It throws a CORS error
(D) It overwrites the request URL

76. What is the exact execution order of the following snippet: `console.log('1'); setTimeout(() => console.log('2'), 0); Promise.resolve().then(() => console.log('3')).then(() => console.log('4')); console.log('5');`?

(A) '1', '5', '3', '4', '2'
(B) '1', '2', '3', '4', '5'
(C) '1', '5', '2', '3', '4'
(D) '1', '3', '4', '5', '2'

77. Why does Promise chaining execute microtasks '3' and '4' before the timer task '2' in Q76?

(A) Microtasks (Promises) have higher scheduling priority than Macrotasks (setTimeout); the event loop drains the entire microtask queue before picking the next macrotask
(B) setTimeout(0) has a mandatory 100ms delay
(C) Promises compile into CPU hardware interrupts
(D) Macrotasks only run after page reload

78. What is the result of the following async code: `async function test() { console.log('A'); await null; console.log('B'); } test(); console.log('C');`?

(A) 'A', 'C', 'B' (A runs synchronously; 'await' pauses function and queues microtask; synchronous code logs C; then microtask logs B)
(B) 'A', 'B', 'C'
(C) 'C', 'A', 'B'
(D) 'B', 'A', 'C'

79. How do you implement a robust retry mechanism with exponential backoff for a failing Fetch request?

(A) Catch failure, wait for `baseDelay * (2 ** attempt)` ms, and recursively retry until reaching maxRetries
(B) Use while(true) loop without delay
(C) Call window.reload()
(D) Increase timeout to 60 seconds

80. What is an 'AggregateError' in JavaScript, and when is it instantiated?

(A) A specialized Error subclass grouping multiple independent errors together, thrown by `Promise.any()` when all input promises reject
(B) An error thrown when JSON parsing fails
(C) A network timeout error
(D) A database connection error

81. What is the difference between `Promise.prototype.then(success, fail)` and `Promise.prototype.then(success).catch(fail)`?

(A) The former cannot catch errors thrown inside its OWN success callback; the chained `.catch()` catches errors thrown in both the promise AND the preceding success handler
(B) They are 100% functionally identical
(C) The former is faster
(D) The latter ignores network errors

82. What happens if a Promise executor function contains both `resolve('val')` and `reject('err')` sequentially?

(A) The first call wins (`resolve('val')`); subsequent state change calls are permanently ignored because Promises settle at most once
(B) It throws a TypeError
(C) It switches to rejected
(D) Both callbacks fire

83. How does the browser handle CORS credentials when `Access-Control-Allow-Origin: *` (wildcard) is returned by the server?

(A) The browser strictly blocks the response and throws a CORS error if `credentials: 'include'` is set; credentials mandate an explicit origin
(B) The browser accepts cookies anyway
(C) The browser caches the response
(D) The browser encrypts the cookie

84. What is the purpose of the `Access-Control-Max-Age` response header in CORS negotiations?

(A) Specifies how many seconds the browser is permitted to cache the preflight OPTIONS response, avoiding repeated preflights for subsequent requests
(B) Sets session cookie lifetime
(C) Limits file download duration
(D) Sets WebSocket keepalive ping interval

85. What is Request Deduplication in frontend data fetching architectures?

(A) Caching in-flight HTTP requests so that multiple simultaneous components requesting identical URLs share a single network call rather than duplicating traffic
(B) Compressing duplicate headers
(C) Deleting duplicate cookies
(D) Filtering out duplicate CSS files

86. How can you implement a request timeout with `AbortController` in modern JavaScript Fetch?

(A) const c = new AbortController(); setTimeout(() => c.abort(), 5000); fetch(url, { signal: c.signal });
(B) fetch(url, { timeout: 5000 });
(C) window.setTimeout(fetch, 5000);
(D) fetch.abort(5000);

87. What is Long-Polling in web communications, and how does it emulate real-time push?

(A) The client issues an HTTP request; the server holds the connection open until new data arrives, sends the response, and client immediately opens a new request
(B) The client polls server every 10 milliseconds
(C) The client keeps an open UDP socket
(D) The server pings the client via DNS

88. What does the WebSocket `socket.readyState` value of 1 represent?

(A) OPEN (The connection is established, handshake complete, and ready for communication)
(B) CONNECTING
(C) CLOSING
(D) CLOSED

89. How do WebSockets handle network disconnects or dropped connections automatically?

(A) They do not: developers must implement custom heartbeat ping/pong mechanisms and reconnection loops in JavaScript
(B) WebSockets reconnect natively via TCP
(C) The browser alerts the user
(D) The OS reboots the network card

90. What is Stale-While-Revalidate (SWR) caching strategy in frontend client-side data fetching?

(A) Returns cached (stale) data immediately to render UI instantly, while asynchronously fetching fresh data in the background to update cache and re-render
(B) Deletes cache every 5 minutes
(C) Blocks UI until network responds
(D) Refreshes page on window focus

91. In jQuery, what is the difference between `$(window).on('load')` and `$(document).ready()`?

(A) `ready()` fires as soon as HTML DOM is parsed; `load()` waits until all external assets (images, stylesheets, frames) have finished downloading
(B) `ready()` waits for images; `load()` does not
(C) `load()` is deprecated in HTML5
(D) They fire simultaneously

92. How do you prevent Cross-Site Scripting (XSS) when inserting dynamic strings into jQuery DOM selections?

(A) Use `$(el).text(data)` instead of `$(el).html(data)`
(B) Use `$(el).append(data)`
(C) Convert to uppercase
(D) Use `$.ajax()` with dataType: 'script'

93. What does the jQuery method `$.noConflict()` achieve when included in a webpage?

(A) Relinquishes jQuery's control of the `$` variable shortcut so other libraries (like Prototype or MooTools) can use it without conflicts
(B) Disables all jQuery animations
(C) Clears browser cache
(D) Enforces strict mode

94. What is an Asynchronous Generator (`async function*`) in ES2018?

(A) A generator function whose iterator `next()` returns a Promise resolving to `{ value, done }`, consumed using `for await...of` loops
(B) A generator running in a Web Worker
(C) A recursive Promise chain
(D) An XMLHttpRequest wrapper

95. How do you consume a stream of asynchronous data using the modern `for await...of` loop?

(A) for await (const chunk of asyncStream) { process(chunk); }
(B) for (const chunk in asyncStream) { await chunk; }
(C) await asyncStream.forEach();
(D) while (asyncStream) { await next(); }

96. What does the `queueMicrotask(fn)` global function do in modern JavaScript?

(A) Explicitly queues a microtask callback onto the microtask queue, executing before the event loop returns to the macrotask queue or browser rendering
(B) Creates a Web Worker thread
(C) Executes fn after 1 second
(D) Schedules a macro timer

97. What is the difference between `structuredClone()` and `JSON.parse(JSON.stringify())` when passing data across asynchronous workers?

(A) `structuredClone()` accurately clones circular references, ArrayBuffers, Maps, Sets, and Dates; JSON methods strip/corrupt non-primitive types and fail on cycles
(B) structuredClone only works on strings
(C) JSON methods are 100x faster
(D) There is zero functional difference

98. In the Streams API, what does the `TransformStream` interface do?

(A) Represents a chaining stream that transforms incoming byte chunks from a ReadableStream before feeding them into a WritableStream
(B) Encrypts TCP packets
(C) Converts SVG into canvas
(D) Formats dates

99. What is Server-Sent Events (SSE) auto-reconnection feature, and how does the server maintain sequence state?

(A) If the connection drops, EventSource automatically reconnects after a delay and transmits the `Last-Event-ID` HTTP header so server resumes from that event
(B) It requires WebSocket fallback
(C) It stores events in localStorage
(D) It reboots the web server

100. Why is an unhandled Promise rejection considered dangerous in modern Node.js and browser environments?

(A) It indicates an unhandled error state that can leave application memory corrupted, and in modern environments triggers fatal process termination or strict console alerts
(B) It deletes database tables
(C) It overflows CPU caches
(D) It causes syntax compilation errors

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Asynchronous Programming in JavaScript, detailing why non-blocking asynchronous execution is vital for single-threaded web browsers.

Q2. Analyze the evolution of asynchronous JavaScript paradigms: Callbacks, Promises, and Async/Await.

Q3. Explain the Promise lifecycle, internal states (Pending, Fulfilled, Rejected), and Promise chaining mechanics.

Q4. Compare Promise concurrency methods: Promise.all(), Promise.allSettled(), Promise.race(), and Promise.any() with engineering use cases.

Q5. Explain the Fetch API in comprehensive detail, including Request options, Response methods, headers, and error handling.

Q6. Explain the Same-Origin Policy (SOP) and Cross-Origin Resource Sharing (CORS) protocol, detailing Simple Requests and Preflights.

Q7. Explain the WebSocket protocol and API architecture, contrasting it with HTTP Polling, Long-Polling, and Server-Sent Events (SSE).

Q8. Describe the architecture of the jQuery library, explaining the jQuery Object wrapper, DOM selection, and method chaining.

Q9. Explain jQuery AJAX capabilities: $.ajax(), $.get(), $.post(), $.getJSON(), and global AJAX event handlers.

Q10. Explain Request Cancellation using AbortController with the Fetch API, demonstrating timeout implementations.

Q11. Detail Server-Sent Events (SSE) using the EventSource API, explaining data framing, reconnection mechanisms, and event types.

Q12. Compare XMLHttpRequest with the modern Fetch API, highlighting eleven architectural differences and migration advantages.

Q13. Explain Asynchronous Iteration (for await...of) and Asynchronous Generators (async function*), writing a paginated API consumer.

Q14. Describe the Streams API (ReadableStream, WritableStream, TransformStream) and demonstrate reading a streaming response chunk-by-chunk.

Q15. Explain Web Workers concurrency model and demonstrate running CPU-heavy calculations without freezing the UI.

Q16. Explain the Retry Pattern with Exponential Backoff and Jitter for resilient network communication.

Q17. Discuss CORS Security Headers: Access-Control-Allow-Origin, Allow-Credentials, Allow-Methods, Allow-Headers, and Max-Age.

Q18. Detail JSON Web Tokens (JWT) handling in client-side applications, contrasting localStorage vs HttpOnly cookie storage.

Q19. Explain Stale-While-Revalidate (SWR) caching architecture and write an in-memory client-side cache.

Q20. Describe Service Workers and the Cache API for offline progressive web applications (PWA).

Q21. Write a complete JavaScript module to establish and maintain a resilient WebSocket connection with automatic heartbeats and exponential reconnection.

Q22. Write a custom Promise polyfill implementation (MyPromise) in vanilla JavaScript supporting then, catch, and state transitions.

Q23. Implement an asynchronous Promise pool / concurrency throttle that limits parallel execution of tasks to at most N active promises.

Q24. Write a complete AJAX file upload widget using Fetch API and FormData with an interactive progress indicator.

Q25. Demonstrate how to implement an Autocomplete Search Input with debounce, cached queries, and aborted in-flight requests.

Q26. Explain how jQuery plugins are authored, writing a custom jQuery plugin `$.fn.tooltip()` with configurable options.

Q27. Explain the IndexedDB API and write a helper class to store and retrieve offline key-value records asynchronously.

Q28. Analyze Security Vulnerabilities in AJAX applications: Cross-Origin Data Leakage, JSON Hijacking, and CSRF attacks.

Q29. Explain the Beacon API (`navigator.sendBeacon()`) and contrast it with Fetch on page unload.

Q30. Analyze Network Waterfall diagnostics in Browser Developer Tools: Queuing, DNS Lookup, Initial Connection, TTFB, and Content Download.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

is vital for single-threaded web

The Three Mutual Exclusive States:
1. Pending: Initial state; operation is actively underway. Has neither resolved nor rejected.
2. Fulfilled (Resolved): Operation completed successfully; holds a permanent resulting `value`.
3. Rejected: Operation failed; holds an associated `reason` (typically an Error object).
Once a Promise transitions from Pending to either Fulfilled or Rejected, it is 'Settled' and permanently locked (immutable).

Promise Chaining Mechanics:
- Every `.then()` or `.catch()` call returns a BRAND NEW Promise.
- If a callback returns a value, the new Promise resolves with that value.
- If a callback returns another Promise, the new Promise adopts the state and value of that returned Promise.
- If a callback throws an exception, the returned Promise immediately transitions to Rejected.

```
method: 'POST',
headers: {
'Content-Type': 'application/json',
'Authorization': 'Bearer ' + token
},
body: JSON.stringify(payload),
mode: 'cors',
credentials: 'include' // Transmit cookies cross-origin
};

try {
const response = await fetch(url, options);
// Critical: Fetch does NOT reject on 4xx/5xx HTTP errors!
if (!response.ok) {
throw new Error(`HTTP error! Status: ${response.status} ${response.statusText}`);
}
const data = await response.json();
return data;
} catch (err) {
console.error('Network request failed:', err);
throw err;
}
}
```

Response Body Extractors: `response.json()` (parses JSON), `response.text()` (plain text), `response.blob()` (binary images/media), and `response.formData()`.

>>Solution to Question 6: Explain the Same-Origin Policy (SOP) and Cross-Origin Resource Sharing (CORS) protocol, detailing Simple Requests and Preflights.

request; otherwise, it throws a CORS error.

```javascript
$.ajax({
url: '/api/items',
type: 'POST',
data: JSON.stringify({ name: 'Widget', qty: 5 }),
contentType: 'application/json; charset=utf-8',
dataType: 'json',
timeout: 5000,
success: function(response) { console.log('Created item:', response); },
error: function(xhr, status, error) { console.error('AJAX Error:', error); }
});
```

2. Shorthand Methods:
- `$.get(url, params, callback)`: Rapid GET queries.
- `$.post(url, data, callback)`: Rapid POST submissions.
- `$.getJSON(url, callback)`: Automatically parses incoming JSON payload.

3. Global AJAX Event Handlers:
`$(document).ajaxStart(() => $('#spinner').show());` and `$(document).ajaxStop(() => $('#spinner').hide());` manage global loading spinners across all network calls.

>>Solution to Question 10

Explain Request Cancellation using AbortController with the Fetch API, demonstrating timeout implementations.

>>Solution Question 12: Server-Sent Events (SSE) is another technology that allows a server to push data to a client over a single HTTP connection. Unlike WebSockets, SSE is unidirectional, meaning data flows only from the server to the client. SSE is simpler to implement and is well-suited for applications that require real-time updates, such as live feeds or notifications.

9. CORS: Both support CORS, but Fetch enforces cleaner origin controls.
10. Memory footprint: Fetch uses structured response objects that stream efficiently.
11. Adoption: Fetch is standard across modern browsers and Node.js 18+.

```
if (done) break;
const textChunk = decoder.decode(value, { stream: true });
fullText += textChunk;
updateLiveTerminal(textChunk); // Render text incrementally as it arrives!
}
console.log('Stream fully finished.');
}
```

Crucial for AI chat completions (ChatGPT streaming), large file processing, and video feeds.

>>Solution Question 16 Main UI Worker Pattern with Exponential Backoff and Jitter for resilient network calls without freezing the UI.

```
async function fetchWithRetry(url, options = {}, retries = 3, baseDelay = 1000) {
for (let attempt = 0; attempt <= retries; attempt++) {
try {
const res = await fetch(url, options);
if (res.status === 429 || res.status >= 500) {
throw new Error(`Transient server error: ${res.status}`);
}
return await res.json();
} catch (err) {
if (attempt === retries) throw err;
// Exponential backoff: 2^attempt + random jitter (0-200ms)
const jitter = Math.random() * 200;
const delay = (baseDelay * (2 ** attempt)) + jitter;
console.warn(`Attempt ${attempt + 1} failed. Retrying in ${Math.round(delay)}ms...`);
await new Promise(r => setTimeout(r, delay));
}
}
}
```

>>Solution Question 14: CORS Server Token (JWT) Access-Control-Allow-Origin Allow-Credentials Allow-Methods Allow-Headers Max-Age.

session token.

2. Storage in `HttpOnly` Secure Cookies:
- Advantage: The browser completely prevents JavaScript from accessing the cookie (`document.cookie` cannot read it), neutralizing XSS token theft.
- Tradeoff: Vulnerable to Cross-Site Request Forgery (CSRF). Requires configuring `SameSite=Strict` (or `Lax`) cookie attributes and implementing Anti-CSRF synchronizer tokens.

>>Solution to Question 28: Service Worker API for offline progressive web applications (PWAs) cache.

```
// Inside sw.js:
self.addEventListener('fetch', (event) => {
event.respondWith(
caches.match(event.request).then((cachedResponse) => {
// Return cached file if found; else fallback to network fetch
return cachedResponse || fetch(event.request);
})
);
});
```

Enables web applications to launch instantly and operate seamlessly with zero internet connectivity.

>> Solution to Question 21: Here is a complete JavaScript module to establish and maintain a resilient WebSocket connection with automatic heartbeats and exponential backoff reconnection:

```
  scheduleReconnect() {
  const delay = Math.min(1000 * (2 ** this.reconnectAttempts), 30000);
  console.log(`Reconnecting in ${delay}ms...`);
  setTimeout(() => {
  this.reconnectAttempts++;
  this.connect();
  }, delay);
  }
  }
  ```
```

---

**Solution to Question 22:** a custom Promise polyfill implementation (MyPromise) in vanilla JavaScript supporting then, catch, and state transitions.
```

```
  }
  };
  handle();
  });
  }
  catch(onRejected) { return this.then(null, onRejected); }
  }
  ```

>>Solution Question 28 ... Promise.race(iterable) ... Fetch API ... Data with ... tasks ... N active promises.

```
  return new Promise((resolve, reject) => {
  const xhr = new XMLHttpRequest();
  xhr.open('POST', '/api/upload');

  xhr.upload.onprogress = (e) => {
  if (e.lengthComputable) {
  const percent = Math.round((e.loaded / e.total) * 100);
  progressBar.value = percent;
  progressBar.textContent = `${percent}%`;
  }
  };

  xhr.onload = () => {
  if (xhr.status >= 200 && xhr.status < 300) resolve(JSON.parse(xhr.responseText));
  else reject(new Error('Upload failed'));
  };
  xhr.onerror = () => reject(new Error('Network error during upload'));
  xhr.send(formData);
  });
  }
```

>> Solution to Question 25:

Here is a complete, production-ready implementation of an Autocomplete Search Input with debounce, cached queries, and aborted in-flight requests.

```
  renderResults(items);
  } catch (err) {
  if (err.name !== 'AbortError') console.error('Search error:', err);
  }
  }, 300));
  ```
```

---



**>>Solution Question 28** In IndexedDB, an [illegible] is a [illegible] that [illegible] a [illegible] value with [illegible] and [illegible].
```

IndexedDB Helper Implementation:

```javascript
class LocalDB {
constructor(dbName = 'AppDB', storeName = 'records') {
this.dbName = dbName;
this.storeName = storeName;
}

async open() {
return new Promise((resolve, reject) => {
const req = indexedDB.open(this.dbName, 1);
req.onupgradeneeded = (e) => e.target.result.createObjectStore(this.storeName, { keyPath: 'id'
});
req.onsuccess = () => resolve(req.result);
req.onerror = () => reject(req.error);
});
}

async put(record) {
const db = await this.open();
return new Promise((resolve, reject) => {
const tx = db.transaction(this.storeName, 'readwrite');
const req = tx.objectStore(this.storeName).put(record);
req.onsuccess = () => resolve();
req.onerror = () => reject(req.error);
});
}
}
```

>> Solution to Question 28: Analyze Security Vulnerabilities in AJAX applications: Cross-Origin Data Leakage, JSON Hijacking, and CSRF attacks.

[Problem Statement]: Explain the Beacon API (`navigator.sendBeacon()`) and contrast it with Fetch on page unload.
[Detailed Step-by-Step Answer]:
Transmitting telemetry data (analytics, metrics) when a user closes a browser tab was historically problematic:

The Problem with Fetch/XHR on Unload:
When an `unload` or `beforeunload` event fires, standard asynchronous `fetch()` requests are often aborted immediately by the browser before headers transmit. Using synchronous XHR froze page closing, frustrating users.

The Beacon API Solution:
`navigator.sendBeacon('/api/analytics', JSON.stringify(metrics));`

Operational Advantages:
- Asynchronous & Non-Blocking: Does not delay page teardown.
- Browser-Managed Delivery: The browser guarantees transmission in the background even after the tab or process has terminated.
- Uses HTTP POST automatically.

>> Solution to Question 30: Analyze Network Waterfall diagnostics in Browser Developer Tools: Queuing, DNS Lookup, Initial Connection, TTFB, and Content Dow